SIMATS ENGINEERING

ASSESSMENT TOOL 2 — ANSWER KEY

Course: Database Management Systems (CSA05)

Course Outcome: CO5 — Query processing, optimization, and transaction management (BL3, BL4)

Activity Tool: Pseudocode Writing Task (Algorithm Design) | Weightage: 35%

Total Marks: 50 (10 × 5 marks)

Code of Conduct

I P Shashank Reg. No: **192524282** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P Shashank

Each answer below gives the pseudocode algorithm, a plain-language explanation, and an executed demonstration (Python) illustrating the algorithm's behaviour, shown as a terminal screenshot.

> **Q1.** *Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.*
>
> *BL3 – Apply | Marks: 5*

Pseudocode

```
ALGORITHM QueryProcessingPipeline(sql_string):
  1. tree      <- PARSE(sql_string)          // lexical + syntax analysis -> parse tree
  2. tree      <- VALIDATE(tree)             // check tables/columns exist in catalog, type-check
  3. logicalPlan  <- TRANSLATE(tree)           // parse tree -> relational algebra expression
  4. candidatePlans <- GENERATE_PLANS(logicalPlan)   // enumerate equivalent access/join plans
  5. bestPlan  <- OPTIMIZE(candidatePlans)     // cost-based selection of cheapest plan
  6. result    <- EXECUTE(bestPlan)            // execution engine runs the chosen plan
  7. OUTPUT(result)                            // return result set to the caller
END ALGORITHM
```

Explanation

The pipeline has five conceptual stages: a parser turns the SQL text into a parse tree and checks its syntax; a validator confirms every referenced table/column exists and types are compatible; the tree is translated into a logical (relational-algebra) plan; the optimizer generates and costs several equivalent physical plans and picks the cheapest; and the execution engine finally runs that plan and streams the output rows back to the user. Each stage's output is the next stage's input, so an error caught early (e.g. at validation) avoids wasted optimization/execution work.

Executed Demo

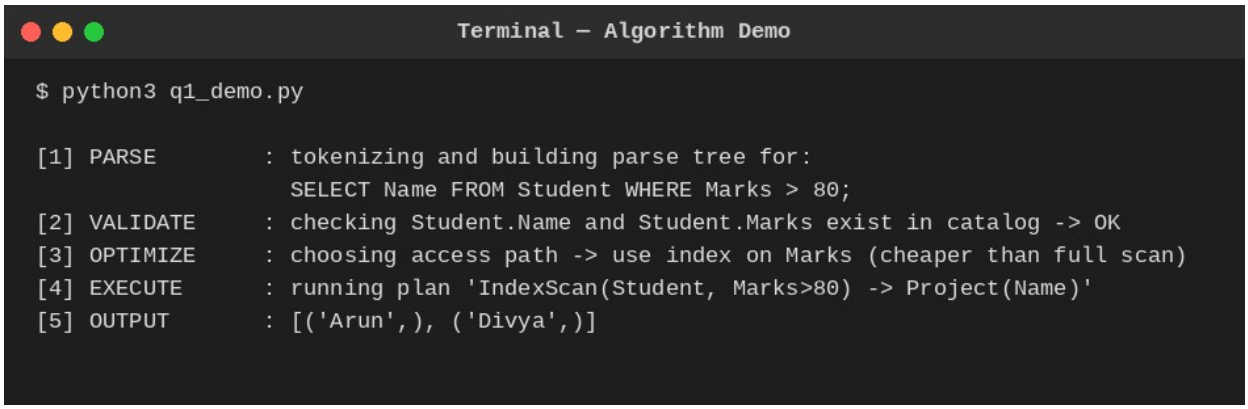

Fig. Q1 — Query processing pipeline simulated stage by stage.

Q2. *Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.*

BL4 – Analyze | Marks: 5

Pseudocode

```
ALGORITHM CostBasedJoinOrdering(relations[], stats[]):
  // stats[r] gives estimated page count of relation r
  1. orderings <- GENERATE_ALL_JOIN_ORDERS(relations)   // or use DP over subsets for larger n
  2. bestCost  <- INFINITY ;  bestOrder <- NULL
  3. FOR EACH order IN orderings:
  4.     runningCost <- 0 ;  intermediateSize <- stats[order[0]]
  5.     FOR i = 1 TO length(order)-1:
  6.         pagesNext <- stats[order[i]]
  7.          joinCost  <- intermediateSize + intermediateSize * pagesNext    // nested-loop I/O
estimate
  8.         runningCost <- runningCost + joinCost
  9.         intermediateSize <- joinCost   // approx. size of the new intermediate result
 10.     IF runningCost < bestCost:
 11.         bestCost <- runningCost ;  bestOrder <- order
 12. RETURN bestOrder, bestCost
END ALGORITHM
```

Explanation

For each candidate join order, the algorithm walks left to right, accumulating an I/O cost estimate using a simple nested-loop cost model (outer pages + outer pages × inner pages) and treating the previous join's cost as the size of the next outer input. It keeps the cheapest ordering seen. Real optimizers use dynamic programming over subsets of relations to avoid the factorial blow-up of trying every permutation, but the per-order cost estimation logic shown here is the same building block.

Executed Demo

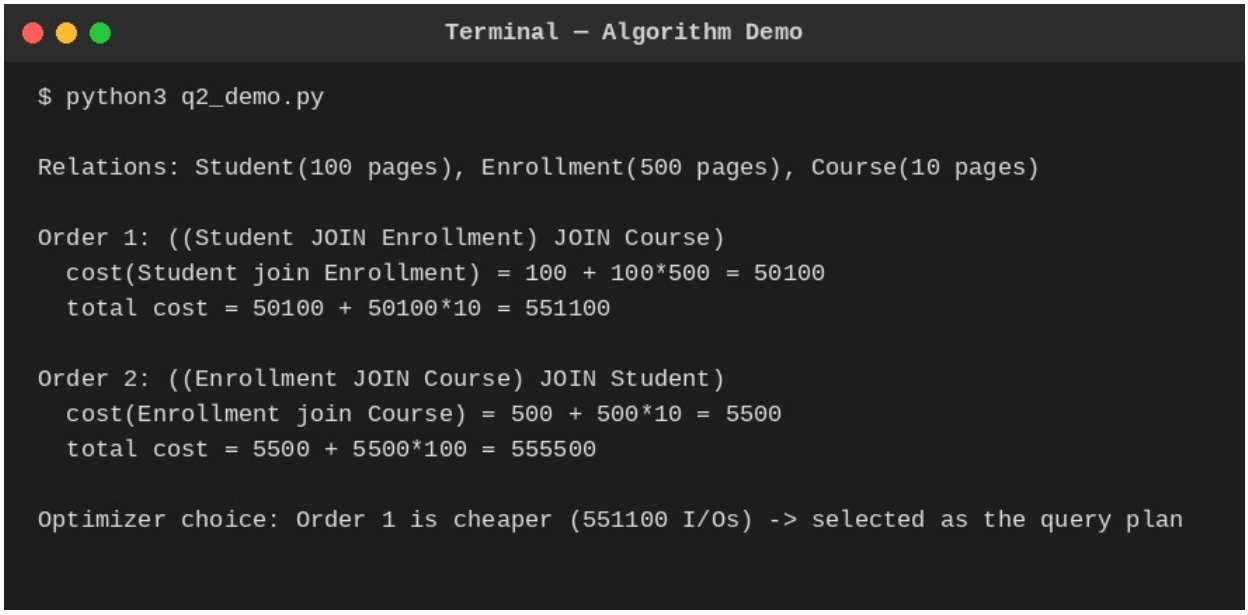

Fig. Q2 — Two join orderings costed and compared.

Q3. *Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.*

BL3 – Apply | Marks: 5

Pseudocode

```
ALGORITHM NestedLoopJoin(R, S, condition):
  result <- {}
  FOR EACH tuple r IN R:            // outer relation, scanned once fully
      FOR EACH tuple s IN S:        // inner relation, re-scanned for every r
          IF condition(r, s) THEN
              result <- result UNION {r JOIN s}
  RETURN result
END ALGORITHM

ALGORITHM BlockNestedLoopJoin(R, S, B):   // B = buffer pages available
  FOR EACH block of (B-2) pages of R:
      FOR EACH block of S:
          FOR EACH r in R-block, s in S-block:
              IF condition(r,s) THEN emit(r JOIN s)
END ALGORITHM
```

Explanation

Tuple-level time complexity is O(|R| × |S|): every outer tuple is compared against every inner tuple, so with |R| and |S| tuples the number of comparisons is their product. In terms of buffer pages, the block-based variant reads the outer relation in chunks of (B−2) pages (reserving one page for the inner scan and one for output), giving I/O cost B(R) + ⌈B(R)/(B−2)⌉ × B(S) block transfers — far fewer disk reads than a page-by-page nested loop, though the tuple-comparison count stays the same order of magnitude.

Executed Demo

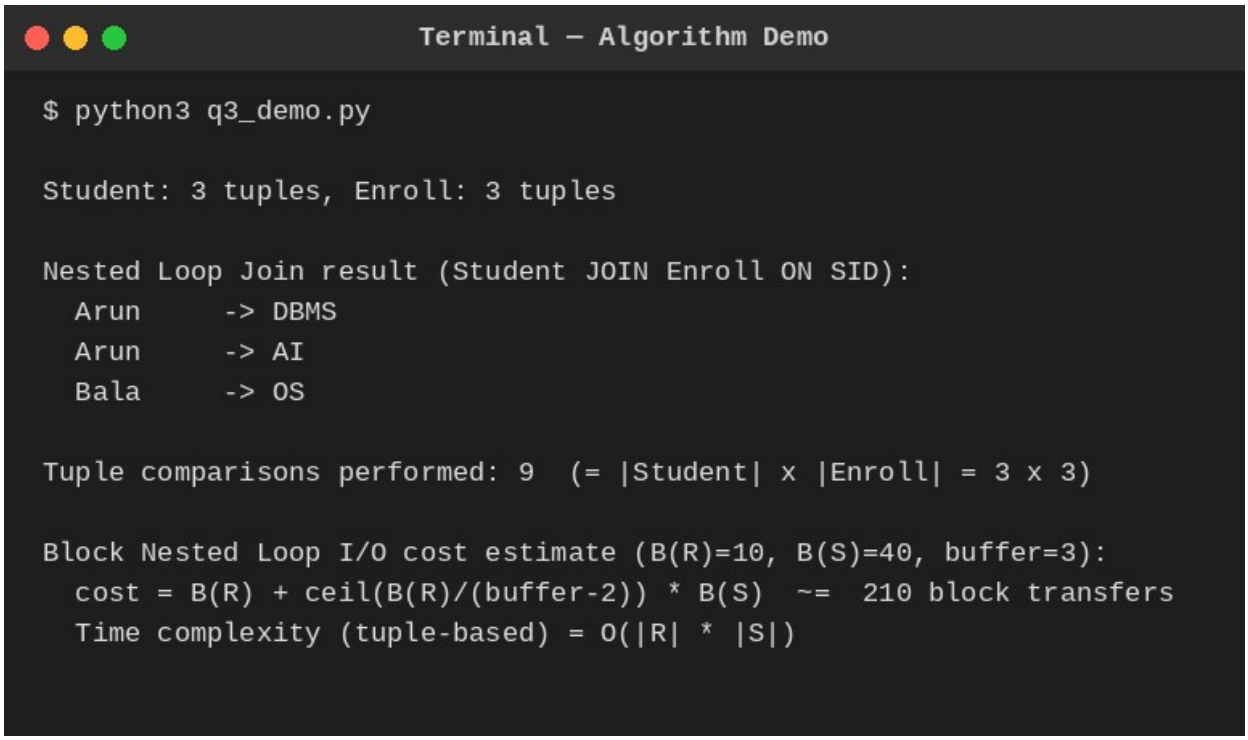

Fig. Q3 — Nested Loop Join executed with comparison and I/O cost counts.

> **Q4.** *Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.*
>
> *BL4 – Analyze | Marks: 5*

Pseudocode

```
ALGORITHM DetectDeadlock(WFG):        // WFG: node = transaction, edge Ti->Tj = Ti waits for Tj
  color <- map each node to WHITE
  FOR EACH node IN WFG:
      IF color[node] = WHITE THEN
          IF DFS_VISIT(node, WFG, color) = TRUE THEN
              RETURN "DEADLOCK DETECTED"
  RETURN "NO DEADLOCK"

FUNCTION DFS_VISIT(node, WFG, color):
  color[node] <- GRAY                 // on the current recursion stack
  FOR EACH neighbor IN WFG[node]:
      IF color[neighbor] = GRAY THEN RETURN TRUE     // back-edge -> cycle found
      IF color[neighbor] = WHITE AND DFS_VISIT(neighbor, WFG, color) THEN RETURN TRUE
  color[node] <- BLACK                // fully explored, no cycle through here
  RETURN FALSE
END ALGORITHM
```

Explanation

The scheduler maintains a Wait-For Graph where an edge Ti → Tj means Ti is blocked waiting for a lock held by Tj. A depth-first search that colors nodes GRAY while on the current path and BLACK once fully explored detects a cycle the moment it revisits a GRAY node — that back-edge is proof of a circular wait, i.e. a deadlock. Once found, the system picks a victim transaction from the cycle (e.g. the youngest, or the one with least work done) and aborts it to break the cycle.

Executed Demo

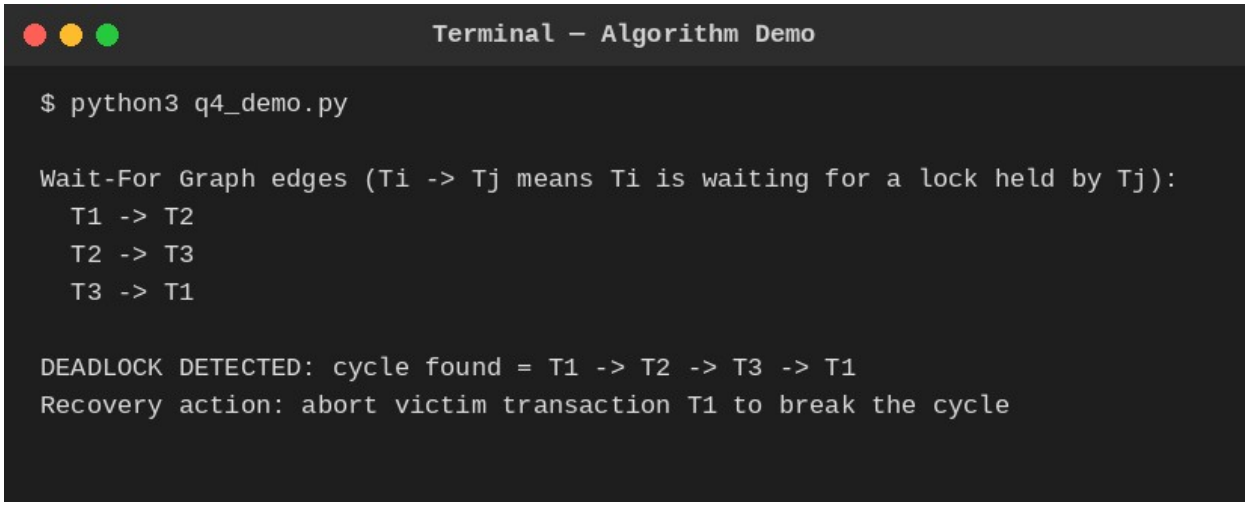

Fig. Q4 — Cycle detected in a 3-transaction Wait-For Graph.

Q5. *Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.*

BL3 – Apply | Marks: 5

Pseudocode

```
ALGORITHM Basic2PL(transaction T):
  phase <- GROWING
  FOR EACH data item d that T needs to access:
      REQUEST_LOCK(T, d, mode)            // mode = Shared or Exclusive
      IF lock on d conflicts with an existing lock held by another txn THEN
          T WAITS until the conflicting lock is released
      GRANT_LOCK(T, d, mode)               // T may now read/write d
      // T must not release ANY lock while still in GROWING phase
  // -- once T releases its first lock, it enters SHRINKING phase --
  phase <- SHRINKING
  FOR EACH lock held by T:
      RELEASE_LOCK(T, d)                   // no new locks may be acquired from here on
END ALGORITHM
```

Explanation

2PL splits a transaction's lifetime into two phases: a growing phase in which it may only acquire locks (never release), and a shrinking phase in which it may only release locks (never acquire). This guarantees conflict-serializable schedules. If a requested lock conflicts with one already held by another active transaction, the requester waits in a queue until it is released; on release, waiting requests are re-evaluated and granted if compatible.

Executed Demo

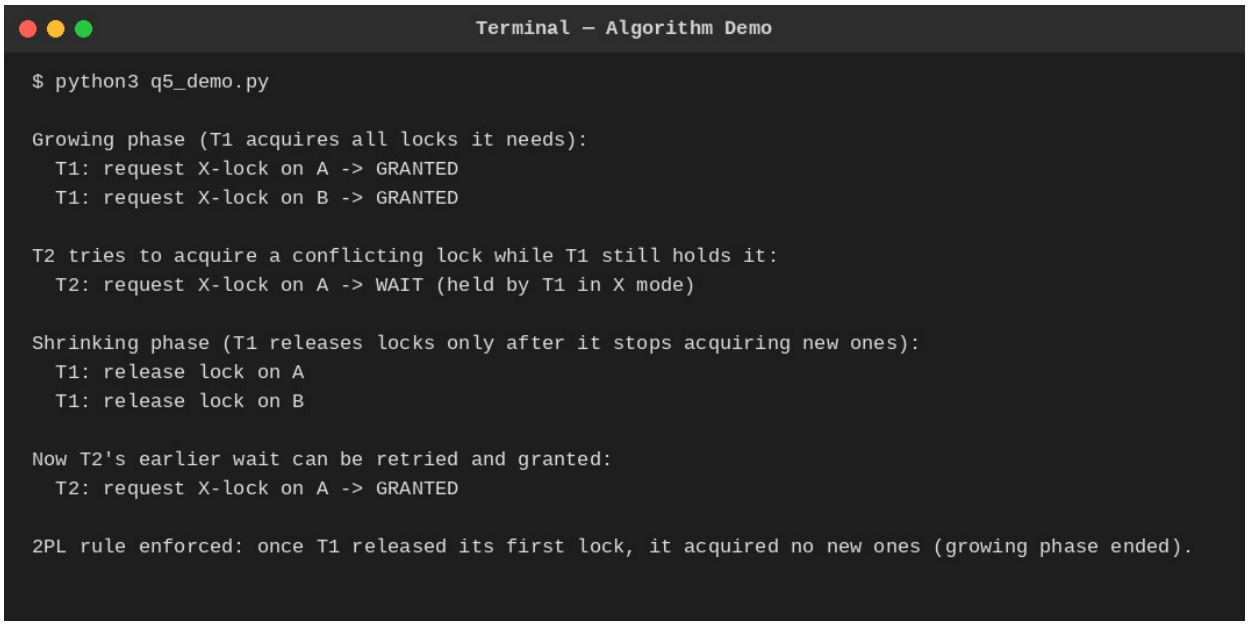

Fig. Q5 — Lock manager simulating growing/shrinking phases for two transactions.

> **Q6.** *Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.*
>
> *BL4 – Analyze | Marks: 5*

Pseudocode

```
ALGORITHM ThomasWriteRule(item, txn, ts):    // ts = TS(txn); item has R-TS, W-TS
  ON READ(item, txn):
      IF ts < item.W_TS THEN
          ABORT(txn)                          // trying to read a value overwritten by a younger
txn
      ELSE
          item.R_TS <- MAX(item.R_TS, ts)
          RETURN item.value

  ON WRITE(item, txn, newValue):
      IF ts < item.R_TS THEN
          ABORT(txn)                            // a younger txn already read the item; write is
too late
      ELSE IF ts < item.W_TS THEN
          IGNORE the write                     // *** Thomas' Write Rule ***
          // a younger write already committed a newer version -- silently skip, do NOT abort
      ELSE
          item.value <- newValue
          item.W_TS  <- ts
END ALGORITHM
```

Explanation

Basic timestamp ordering would abort any write that arrives with an older timestamp than the item's current W-TS. Thomas' Write Rule relaxes this: if no transaction has read the stale value in between, that obsolete write can simply be ignored instead of aborting the transaction, since its effect would have been overwritten anyway. This reduces unnecessary rollbacks while still producing a result equivalent to some serial order.

Executed Demo

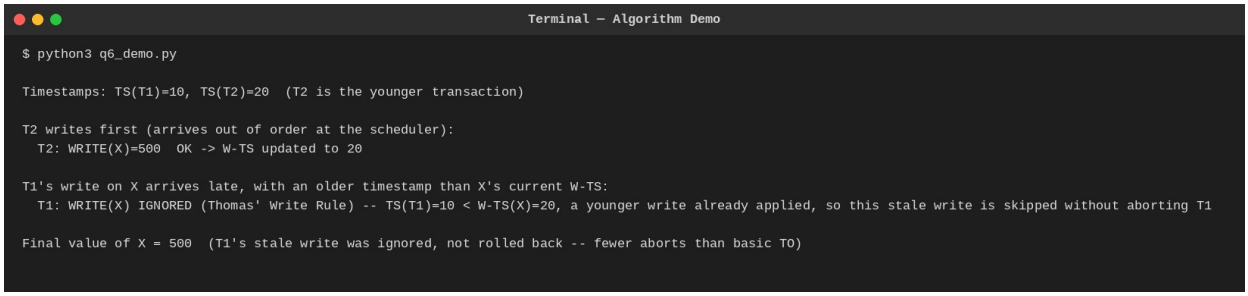

Fig. Q6 — Younger write applied first; an older, stale write is ignored per Thomas' Rule.

> **Q7.** *Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.*
>
> *BL3 – Apply | Marks: 5*

Pseudocode

```
ALGORITHM ARIESRecovery(log):
  // -- ANALYSIS PHASE --
  scan log forward from the last checkpoint
  FOR EACH record IN log:
      IF record.type = update THEN add record.page to DirtyPageTable
                                  add record.txn to TransactionTable (active)
      IF record.type = commit THEN remove record.txn from TransactionTable
  losers <- transactions still in TransactionTable   // uncommitted at crash

  // -- REDO PHASE --
  scan log forward from the oldest relevant LSN in DirtyPageTable
  FOR EACH update record IN log (ALL of them, committed or not):
      REDO the update on record.page   // repeat history exactly as it happened

  // -- UNDO PHASE --
  scan log backward from the end
  FOR EACH update record belonging to a loser transaction:
      UNDO the update (restore before-image)
      WRITE a Compensation Log Record (CLR) so this undo is never redone again
  mark all losers as ABORTED
END ALGORITHM
```

Explanation

ARIES recovers in three passes. Analysis rebuilds the state at crash time — which pages were dirty and which transactions were still active (the losers). Redo then repeats history: it reapplies every logged update, committed or not, bringing the database back to exactly the state it was in at the moment of the crash. Undo then rolls back only the losers' updates, in reverse order, writing a CLR for each so that a second crash during recovery never re-undoes the same action.

Executed Demo

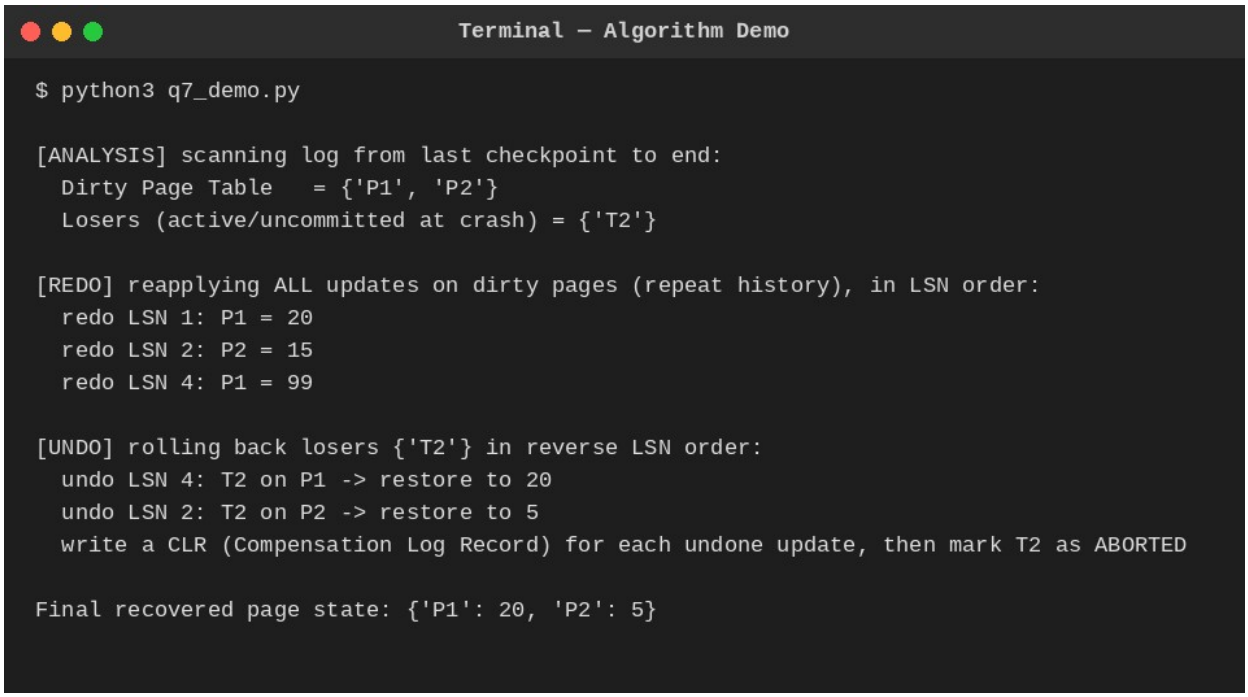

Fig. Q7 — Analysis, Redo, and Undo phases executed over a sample log.

> **Q8.** *Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.*
>
> *BL3 – Apply | Marks: 5*

Pseudocode

```
ALGORITHM ShadowPaging(transaction T):
  shadowTable  <- COPY of the last committed page table
  currentTable <- COPY of shadowTable          // T works on this copy

  FOR EACH write(page p, newValue) requested by T:
      newFrame <- ALLOCATE a fresh page frame on disk
      WRITE newValue INTO newFrame
      currentTable[p] <- newFrame               // copy-on-write; shadowTable[p] untouched

  IF T reaches COMMIT THEN
      FLUSH currentTable and all new frames to disk
      ATOMICALLY set shadowTable <- currentTable    // single pointer swap = the commit point
  ELSE IF system CRASHES or T ABORTS before commit THEN
      DISCARD currentTable and any new frames
      shadowTable remains the valid, consistent database (no undo needed)
END ALGORITHM
```

Explanation

Shadow paging never overwrites data in place. Every write allocates a brand-new page frame, and only the current page table's pointer is updated to reference it — the shadow (old) page table still points to the original, untouched frames. Commit is a single atomic pointer swap (shadow := current), which is why recovery needs no REDO/UNDO log processing at all: on a crash before commit, the old shadow table is simply still valid, and the half-written current table and its new frames are just discarded as garbage.

Executed Demo

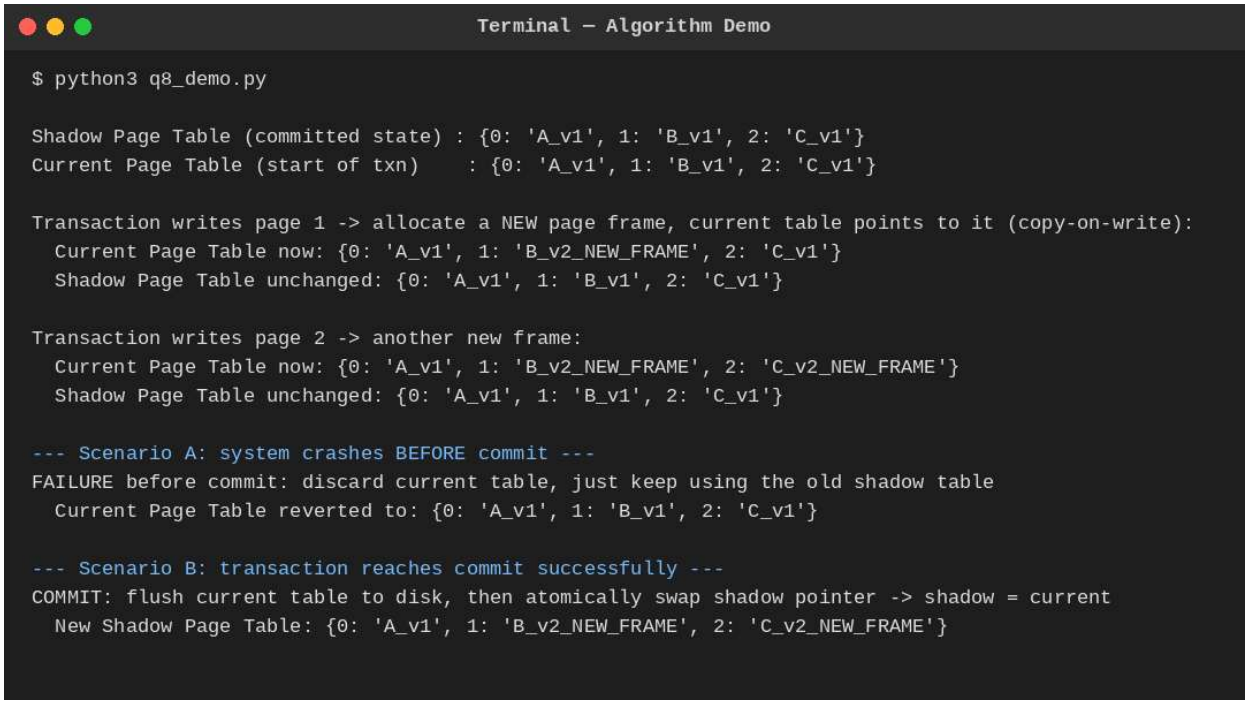

Fig. Q8 — Copy-on-write pages, commit swap, and pre-commit crash rollback.

Q9. *Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.*
BL3 – Apply | Marks: 5

Pseudocode

```
ALGORITHM ImmediateUpdateRecovery(log):
  committed   <- {} ;  active <- {}
  FOR EACH record IN log:                       // pass 1: classify transactions
      IF record.type = start  THEN active.add(record.txn)
      IF record.type = commit THEN active.remove(record.txn); committed.add(record.txn)
  uncommitted <- active                          // never reached commit before the crash

  // REDO pass: forward through the log
  FOR EACH write record IN log (in order):
      IF record.txn IN committed THEN
          REDO: item[record.item] <- record.afterValue

  // UNDO pass: backward through the log
  FOR EACH write record IN log (reverse order):
      IF record.txn IN uncommitted THEN
          UNDO: item[record.item] <- record.beforeValue
END ALGORITHM
```

Explanation

Because immediate update writes changes to disk as soon as they happen (before commit), a crash can leave both committed and uncommitted effects on disk in an unpredictable mix. Recovery therefore needs both directions: REDO reapplies every write belonging to a transaction that did commit (in case its update didn't make it to disk before the crash), and UNDO reverses every write belonging to a transaction that never committed (in case it did make it to disk). The before/after images stored in each log record make both

operations possible.

Executed Demo

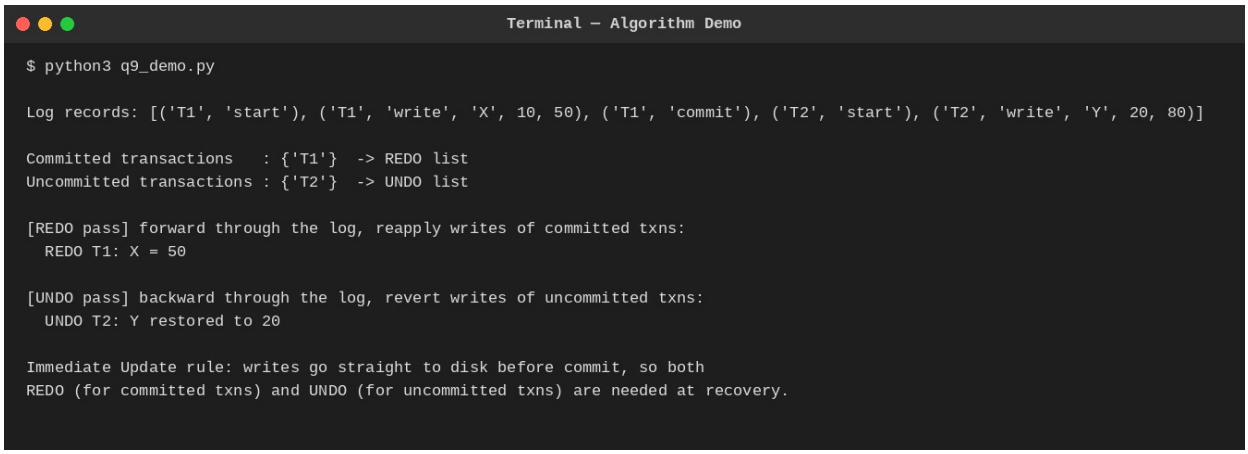

Fig. Q9 — REDO of a committed transaction and UNDO of an uncommitted one.

> **Q10.** *Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.*
>
> *BL4 – Analyze | Marks: 5*

Pseudocode

```
ALGORITHM ConnectionPoolManager(poolSize):
  pool <- QUEUE of poolSize pre-opened connections

  FUNCTION ACQUIRE():
      IF pool is empty THEN BLOCK caller until a connection is released
      RETURN pool.dequeue()

  FUNCTION RELEASE(conn):
      pool.enqueue(conn)

  FUNCTION RUN_QUERY(query):
      conn <- ACQUIRE()
      TRY:
          conn.BEGIN_TRANSACTION()
          result <- conn.EXECUTE(query)
          conn.COMMIT()
          RETURN result
      CATCH exception:
          conn.ROLLBACK()                    // undo partial effects of the failed transaction
          LOG the exception
      FINALLY:
          RELEASE(conn)                      // connection always returned, even on failure
END ALGORITHM
```

Explanation

A connection pool avoids the cost of opening a fresh database connection per request by keeping a small set of live connections that callers borrow and return. RUN_QUERY wraps every query in a try/catch/finally: the transaction is begun, executed, and committed on success; on any exception it is rolled back instead so no

partial write is left behind; and the finally block guarantees the connection always goes back to the pool, whether the query succeeded or failed — preventing connection leaks under error conditions.

Executed Demo

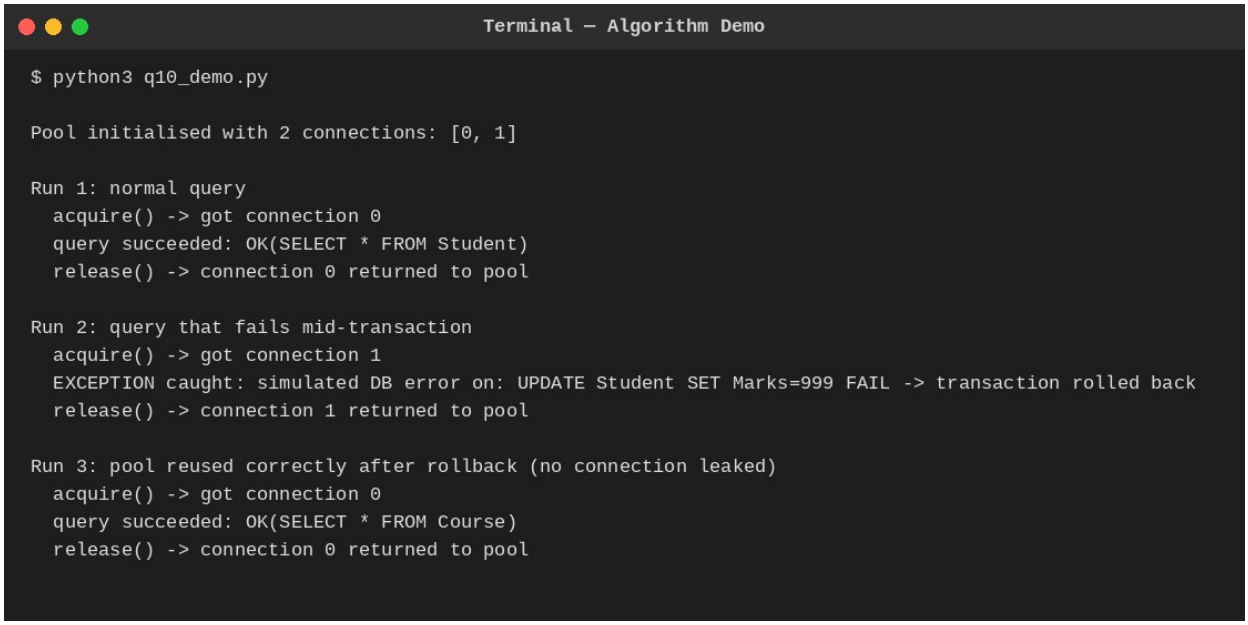

Fig. Q10 — Pool serving a normal query, a failing query with rollback, and reuse afterward.

End of Answer Key